

Detection & Prevention

We are on the downslope now! Let's take a break from our super-spy business of infiltrating hosts and take a look at the defensive side. This section explores ways to detect active shells, look for payloads on a host and over network traffic, and how these attacks can be obfuscated to bypass our defenses.

Monitoring

When it comes to looking for and identifying active shells, payload delivery and execution, and potential attempts to subvert our defenses, we have many different options to utilize to detect and respond to these events. Before talking about data sources and tools we can use, let's take a second to talk about the [MITRE ATT&CK Framework](#) and define the techniques and tactics being utilized by attackers. The [ATT&CK Framework](#) as defined by MITRE, is "a globally-accessible knowledge base of adversary tactics and techniques based on real-world observations."

ATT&CK Framework

Initial Access 8 techniques	Execution 10 techniques	Persistence 10 techniques	Privilege Escalation 12 techniques	Defense Evasion 14 techniques	Credential Access 14 techniques	Discovery 24 techniques	Lateral Movement 9 techniques	Collection 16 techniques	Command and Control 16 techniques	Exfiltration 8 techniques	Impact 13 techniques
Valid Accounts	Application Through Removable Media	Scheduled Task/Job	Valid Accounts	Modify Authentication Process	System Service Discovery	Remote Services	Data from Local System	Data Obfuscation	Exfiltration Over Other	Data Destruction	
Trusted Relationship	Software Deployment	Boot or Logon Initialization Scripts	Direct Volume Access	Input Capture	Discovery	Replication Through	Data from Removable Media	Input Capture	Exfiltration Over C2 Channel	Service Stop	
Supply Chain Compromise	Tools	Create or Modify System Process	Rootkit	Brute Force	System Network Configuration Discovery	Internal Spearphishing	Data Staged	Screen Capture	Exfiltration Over Web Service	Resource Hijacking	
Hardware Additions	Shared Modules	Event Triggered Execution	Obfuscated Files or Information	Process Injection	Exploitation for Credential Access	System Owner/User Discovery	Use Alternate Authentication Material	Clipboard Data	Multi-Stage Channels	Network Denial of Service	
Exploit Public-Facing Application	User Execution	Boot or Logon Autostart Execution	Process Injection	Access Token Manipulation	Steal Web Session Cookie	System Network Connections Discovery	Lateral Tool Transfer	Automated Collection	Ingress Tool Transfer	Endpoint Denial of Service	
Phishing	Exploitation for Client Execution	Account Manipulation	Group Policy Modification	Abuse Elevation Control Mechanism	Unsecured Credentials	Permission Groups	Exploitation of Remote Services	Video Capture	Traffic Signaling	Automated Exfiltration	
External Remote Services	System Services	Office Application Startup	Abuse Elevation Control Mechanism	Indicator Removal on Host	Credentials from Password Stores	Discovery	Remote Service Session Hijacking	Man in the Browser	Dynamic Resolution	Exfiltration Over Alternative Protocol	
Drive-by Compromise	Command and Scripting Interpreter	Create Account	Exploitation for Privilege Escalation	Modify Registry	Steal or Forge Kerberos Tickets	File and Directory Discovery	Man in the Middle	Data from Information Repositories	Non-Standard Port	Transfer Data to Cloud Account	
Native API	Inter-Process Communication	Browser Extensions	Escalation	Trusted Developer Utilities Proxy Execution	Man-in-the-Middle	Peripheral Device Discovery	Browser Bookmark	Archive Collected Data	Protocol Tunneling	Encrypted Channel	
Has sub-techniques		Server Software Component		Traffic Signaling	Network Share Discovery	Network Policy Discovery	Discovery	Data from Network Shared Drive	Non-Application Layer Protocol		
		Pre-OS Boot		Signed Script Proxy Execution	Steal Application Access Token	Process Discovery	Discovery	Data from Cloud Storage Object			
		Compromise Client Software Binary		Rogue Domain Controller	Man-in-the-Middle	System Information Discovery	Virtualization/Sandbox Evasion				
		Implant Container Image		Indirect Command Execution		Account Discovery	Software Dashboard				
				BITS Jobs		System Trust Discovery	Cloud Service Discovery				
				XSL Script Processing		Cloud Service Discovery	Query Registry				
				Template Injection		Remote System Discovery	Software Discovery				
				File and Directory Permissions Modification		Network Service Scanning	Process Discovery				
				Virtualization/Sandbox Evasion		System Information Discovery	System Information Discovery				
				Unused/Unsupported Cloud Regions		Account Discovery	Account Discovery				
				Use Alternate Authentication Material		System Trust Discovery	System Trust Discovery				
				Impair Defenses		Domain Trust Discovery	Domain Trust Discovery				
				Hide Artifacts		Cloud Service Discovery	Cloud Service Discovery				
				Masking/Obfuscation							
				Deobfuscate/Decode Files or Information							
				Signed Binary Proxy Execution							
				Exploitation for Defense Evasion							
				Execution Guardrails							
				Modify Cloud Compute Infrastructure							
				Pre-OS Boot							
				Subvert Trust Controls							

MITRE ATT&CK[®] Enterprise Framework

attack.mitre.org

Keeping the framework in mind, three of the most notable techniques we can tie to Shells & Payloads are listed below in the table with descriptions.

Notable MITRE ATT&CK Tactics and Techniques:

Tactic / Technique	Description
Initial Access	Attackers will attempt to gain initial access by compromising a public-facing host or service such as web Applications, misconfigured services such as SMB or authentication protocols, and/or bugs in a public-facing host that introduce a vulnerability. This is often done on some form of bastion host and provides the attacker with a foothold in the network but not yet full access. For more information on initial access, especially via Web Applications, check out the OWASP Top Ten or read further in the Mitre Att&ck framework.
Execution	This technique depends on code supplied and planted by an attacker running on the victim host. The Shells & Payloads module focuses mainly on this tactic. We utilize many different payloads, delivery methods, and shell scripting solutions to access a host. This can be anything from the execution of commands within our web browser to get execution and access on a Web Application. issuing a PowerShell

one-liner via PsExec, taking advantage of a publicly released exploit or zero-day in conjunction with a framework such as Metasploit, or uploading a file to a host via many different protocols and calling it remotely to receive a callback.

Command & Control

Command and Control (C2) can be looked at as the culmination of our efforts within this module. We gain access to a host and establish some mechanism for continued and/or interactive access via code execution, then utilize that access to perform follow on actions on objectives within the victim network. The use of standard ports and protocols within the victim network to issue commands and receive output from the victim is common. This can appear as anything from normal web traffic over HTTP/S, commands issued via other common external protocols such as DNS and NTP, and even the use of common allowed applications such as Slack, Discord, or MS Teams to issue commands and receive check-ins. C2 can have various levels of sophistication varying from basic clear text channels like Netcat to utilizing encrypted and obfuscated protocols along with complex traffic routes via proxies, redirectors, and VPNs.

Events To Watch For:

- **File uploads:** Especially with Web Applications, file uploads are a common method of acquiring a shell on a host besides direct command execution in the browser. Pay attention to application logs to determine if anyone has uploaded anything potentially malicious. The use of firewalls and anti-virus can add more layers to your security posture around the site. Any host exposed to the internet from your network should be sufficiently hardened and monitored.
- **Suspicious non-admin user actions:** Looking for simple things like normal users issuing commands via Bash or cmd can be a significant indicator of compromise. When was the last time an average user, much less an admin, had to issue the command `whoami` on a host? Users connecting to a share on another host in the network over SMB that is not a normal infrastructure share can also be suspicious. This type of interaction usually is end host to infrastructure server, not end host to end host. Enabling security measures such as logging all user interactions, PowerShell logging, and other features that take note when a shell interface is used will provide you with more insight.
- **Anomalous network sessions:** Users tend to have a pattern they follow for network interaction. They visit the same websites, use the same applications, and often perform those actions multiple times a day like clockwork. Logging and parsing NetFlow data can be a great way to spot anomalous network traffic. Looking at things such as top talkers, or unique site visits, watching for a heartbeat on a nonstandard port (like 4444, the default port used by Meterpreter), and monitoring any remote login attempts or bulk GET / POST requests in short amounts of time can all be indicators of compromise or attempted exploitation. Using tools like network monitors, firewall logs, and SIEMS can help bring a bit of order to the chaos that is network traffic.

Establish Network Visibility

Much like identifying and then using various shells & payloads, **detection & prevention** requires a detailed understanding of the systems and overall network environment you are trying to protect. It's always essential to have good documentation practices so individuals responsible for keeping the environment secure can have consistent visibility of the devices, data, and traffic flow in the environments they administer. Developing & maintaining visual network topology diagrams can help visualize network traffic flow. Newer tools like [netbrain](#) may be good to research as they combine visual diagraming that can be achieved with tools like [Draw.io](#), documentation and remote management. Interactive visual network topologies allow you to interact with the routers, network firewalls, IDS/IPS appliances, switches, and hosts (clients). Tools like this are becoming more common to use as it can be challenging to keep the visibility of the network updated, especially in larger environments that are constantly growing.

Some network device vendors like Cisco Meraki, Ubiquiti, Check Point, and Palo Alto Networks are building layer 7 visibility (like layer 7 of the OSI model) into their network devices and moving the management capabilities to cloud-based network controllers. This means admins log in to a web portal to manage all the network devices in the environment. A visual dashboard is often provided with these cloud-based network controllers making it easier to have a **baseLine** of the traffic usage, network protocols, applications, and inbound & outbound traffic. Having and understanding the baseline of your network will make any deviation from the norm extremely visible. The faster you can react and triage any potential issue, the less time for possible leaks, destruction of data, or worse.

Keep in mind that if a payload is successfully executed, it will need to communicate over the network, so this is why network visibility is essential within the context of shells & payloads. Having a network security appliance capable of **deep packet inspection** can often

is essential within the context of shells & payloads. Having a network security appliance capable of [deep packet inspection](#) can often act as an anti-virus for the network. Some of the payloads we discussed could get detected & blocked at the network level if successfully executed on the hosts. This is especially easy to detect if traffic is not encrypted. When we used Netcat in the bind & reverse shell sections, the traffic passing between the source and destination (target) was **not encrypted**. Someone could capture that traffic and see every command we sent between our attack box and the target, as seen in the examples below.

This image shows NetFlow between two hosts frequently and on a suspicious port (4444). We can tell it is basic TCP traffic, so if we take action and inspect it a bit, we can see what's going on.

Suspicious Traffic.. In Clear Text

The image shows a Wireshark packet capture of a Netcat netcat.pcap file. The packet list shows a series of TCP connections between 10.129.43.29 and 10.129.43.4 on port 4444. The packet details for frame 29 show a netcat reverse shell connection.

No.	Time	Source	Destination	Protocol	src.p	dest.p	Length	Info
13	15.211515	10.129.43.29	10.129.43.4	TCP	50612	4444	64	50612 → 4444 [PSH, ACK]
14	15.211538	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
15	15.261797	10.129.43.29	10.129.43.4	TCP	50612	4444	254	50612 → 4444 [PSH, ACK]
16	15.261833	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
17	15.261986	10.129.43.29	10.129.43.4	TCP	50612	4444	841	50612 → 4444 [PSH, ACK]
18	15.261992	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
19	21.584905	10.129.43.4	10.129.43.29	TCP	4444	50612	61	4444 → 50612 [PSH, ACK]
20	21.626201	10.129.43.29	10.129.43.4	TCP	50612	4444	68	50612 → 4444 [PSH, ACK]
21	21.626254	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
22	22.582605	10.129.43.4	10.129.43.29	TCP	4444	50612	58	4444 → 50612 [PSH, ACK]
23	22.646451	10.129.43.29	10.129.43.4	TCP	50612	4444	60	50612 → 4444 [ACK] Seq=
24	22.646488	10.129.43.29	10.129.43.4	TCP	50612	4444	255	50612 → 4444 [PSH, ACK]
25	22.646503	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
26	22.646648	10.129.43.29	10.129.43.4	TCP	50612	4444	314	50612 → 4444 [PSH, ACK]
27	22.646653	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
28	41.703799	10.129.43.4	10.129.43.29	TCP	4444	50612	85	4444 → 50612 [PSH, ACK]
29	41.720894	10.129.43.29	10.129.43.4	TCP	50612	4444	86	50612 → 4444 [PSH, ACK] Seq=

Frame 29: 86 bytes on wire (688 bits), 86 bytes captured (688 bits)

```

0000 00 50 56 b9 6c 2c 00 50 56 b9 93 48 08 00 45 00  ·P·l·P·V·H·E·
0010 00 48 13 35 40 00 80 06 7c 58 0a 81 2b 1d 0a 81  ·H·5@· ·X·+· ·
0020 2b 04 c5 b4 11 5c af 3b ba f6 86 6d ce 0d 50 18  +· · · · ·\ ; · · · · ·P·
0030 20 14 f2 a0 00 00 6e 65 74 20 75 73 65 72 20 68  · · · · ·ne t user h
0040 61 63 6b 65 72 20 50 61 73 73 77 30 72 64 31 20  acker Pa ssw0rd1
0050 2f 61 64 64 0d 0a  ·/add· ·

```

Notice now that that same traffic has been expanded, and we can see that someone is using **net** commands to create a new user on this host.

Following the Traffic

The image shows a Wireshark packet capture of a Netcat netcat.pcap file. The packet list shows a series of TCP connections between 10.129.43.29 and 10.129.43.4 on port 4444. The packet details for frame 29 show a netcat reverse shell connection.

No.	Time	Source	Destination	Protocol	src.p	dest.p	Length	Info
13	15.211515	10.129.43.29	10.129.43.4	TCP	50612	4444	64	50612 → 4444 [PSH, ACK]
14	15.211538	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
15	15.261797	10.129.43.29	10.129.43.4	TCP	50612	4444	254	50612 → 4444 [PSH, ACK]
16	15.261833	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
17	15.261986	10.129.43.29	10.129.43.4	TCP	50612	4444	841	50612 → 4444 [PSH, ACK]
18	15.261992	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
19	21.584905	10.129.43.4	10.129.43.29	TCP	4444	50612	61	4444 → 50612 [PSH, ACK]
20	21.626201	10.129.43.29	10.129.43.4	TCP	50612	4444	68	50612 → 4444 [PSH, ACK]
21	21.626254	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
22	22.582605	10.129.43.4	10.129.43.29	TCP	4444	50612	58	4444 → 50612 [PSH, ACK]
23	22.646451	10.129.43.29	10.129.43.4	TCP	50612	4444	60	50612 → 4444 [ACK] Seq=
24	22.646488	10.129.43.29	10.129.43.4	TCP	50612	4444	255	50612 → 4444 [PSH, ACK]
25	22.646503	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
26	22.646648	10.129.43.29	10.129.43.4	TCP	50612	4444	314	50612 → 4444 [PSH, ACK]
27	22.646653	10.129.43.4	10.129.43.29	TCP	4444	50612	54	4444 → 50612 [ACK] Seq=
28	41.703799	10.129.43.4	10.129.43.29	TCP	4444	50612	85	4444 → 50612 [PSH, ACK]
29	41.720894	10.129.43.29	10.129.43.4	TCP	50612	4444	86	50612 → 4444 [PSH, ACK] Seq=

Frame 29: 86 bytes on wire (688 bits), 86 bytes captured (688 bits)

```

0000 00 50 56 b9 6c 2c 00 50 56 b9 93 48 08 00 45 00  ·P·l·P·V·H·E·
0010 00 48 13 35 40 00 80 06 7c 58 0a 81 2b 1d 0a 81  ·H·5@· ·X·+· ·
0020 2b 04 c5 b4 11 5c af 3b ba f6 86 6d ce 0d 50 18  +· · · · ·\ ; · · · · ·P·
0030 20 14 f2 a0 00 00 6e 65 74 20 75 73 65 72 20 68  · · · · ·ne t user h
0040 61 63 6b 65 72 20 50 61 73 73 77 30 72 64 31 20  acker Pa ssw0rd1
0050 2f 61 64 64 0d 0a  ·/add· ·

```



```
c:\Users\mrb3n\Downloads>cd c:\
cd c:\

c:\>dir
dir
Volume in drive C has no label.
Volume Serial Number is E8C0-6EAE

Directory of c:\

07/16/2016  04:47 AM    <DIR>          PerfLogs
05/10/2021  01:08 PM    <DIR>          Program Files
05/10/2021  01:08 PM    <DIR>          Program Files (x86)
05/10/2021  07:34 PM    <DIR>          Users
05/10/2021  12:46 PM    <DIR>          Windows
               0 File(s)              0 bytes

               5 Dir(s) 21,421,400,064 bytes free

c:\>net user hacker Passw0rd1 /add
net user hacker Passw0rd1 /add
The command completed successfully.

c:\>net localgroup administrators hacker /add
net localgroup administrators hacker /add
The command completed successfully.

c:\>
```

12 **client** pkts, 6 **server** pkts, 12 turns.

Entire conversation (1914 bytes) Show data as ASCII Stream 0

Find: Find Next

Help Filter Out This Stream Print Save as... Back Close

This is an excellent example of basic access and command execution to gain persistence via the addition of a user to the host.

Regardless of the name **hacker** being used, if command-line logging is in place paired with the NetFlow data, we can quickly tell that the user is performing potentially malicious actions and triage this event to determine if an incident has occurred or if this is just some admin playing around. A modern security appliance may detect, alert and prevent further network communications from that host using deep packet inspection.

Speaking of anti-virus, let's discuss end device detection & protection a bit.

Protecting End Devices

End devices are the devices that connect at the "end" of a network. This means they are either the source or destination of data transmission. Some examples of end devices would be:

- Workstations (employees computers)
- Servers (providing various services on the network)
- Printers
- Network Attached Storage (NAS)
- Cameras
- Smart TVs
- Smart Speakers

We should prioritize the protection of these kinds of devices, especially those that run an operating system with a **CLI** that can be remotely accessed. The same interface that makes it easy to administer and automate tasks on a device can make it a good target for attackers. As simple as this seems, having anti-virus installed & enabled is a great start. The most common successful attack vector besides misconfiguration is the human element. All it takes is for a user to click a link or open a file, and they can be compromised. Having monitoring and alerting on your end devices can help detect and potentially prevent issues before they happen.

On **Windows** systems, **Windows Defender** (also known as Windows Security or Microsoft Defender) is present at install and should be left enabled. Also, ensuring the Defender Firewall is left enabled with all profiles (Domain, Private and Public) left on. Only make exceptions for approved applications based on a **change management process**. Establish a **patch management** strategy (if not already established) to ensure that all hosts are receiving updates shortly after Microsoft releases them. All of this applies to servers hosting shared resources and websites as well. Though it can slow performance, AV on a server can prevent the execution of a payload and the establishment of a shell session with a malicious attacker's system.

Potential Mitigations:

Consider the list below when considering what implementations you can put in place to mitigate many of these vectors or exploits.

- **Application Sandboxing:** By sandboxing your applications that are exposed to the world, you can limit the scope of access and damage an attacker can perform if they find a vulnerability or misconfiguration in the application.
- **Least Privilege Permission Policies:** Limiting the permissions users have can go a long way to help stop unauthorized access or compromise. Does an ordinary user need administrative access to perform their daily duties? What about domain admin? Not really, right? Ensuring proper security policies and permissions are in place will often hinder if not outright stop an attack.
- **Host Segmentation & Hardening:** Properly hardening hosts and segregating any hosts that require exposure to the internet can help ensure an attacker cannot easily hop in and move laterally into your network if they gain access to a boundary host. Following STIG hardening guides and placing hosts such as web servers, VPN servers, etc., in a DMZ or 'quarantine' network segment will stop that type of access and lateral movement.
- **Physical and Application Layer Firewalls:** Firewalls can be powerful tools if appropriately implemented. Proper inbound and outbound rules that only allow traffic first established from within your network, on ports approved for your applications, and denying inbound traffic from your network addresses or other prohibited IP space can cripple many bind and reverse shells. It adds a hop in the network chain, and network implementations such as Network Address Translation (NAT) can break the functionality of a shell payload if it is not taken into account.

Sum It All Up

These protections and mitigations are not the ends all be all for stopping attacks. A strong security posture and defense strategy are required in today's age. Adapting a defense-in-depth approach to your security posture will help hinder attackers and ensure the low-hanging fruit cannot be easily taken advantage of.

[← Previous](#)[✔ Finish](#)[📄 Cheat Sheet](#)

Table of Contents

[Introduction](#)

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

 Anatomy of a Shell



 Bind Shells



 Reverse Shells



Payloads

Introduction to Payloads



 Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom



Windows Shells

 Infiltrating Windows



NIX Shells

 Infiltrating Unix/Linux



Spawning Interactive Shells



Web Shells

Introduction to Web Shells



 Laudanum, One Webshell to Rule Them All



 Antak Webshell



 PHP Web Shells



Skills Assessment

 The Live Engagement



Additional Considerations

[Detection & Prevention](#)

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left